CSA1701-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Program 1 : Python Program to Solve 8. Puzzle Problem

Aim

To develop a Python program that solves the 8-Puzzle Problem using a search algorithm (such as Breadth-First Search - BFS) to find the shortest sequence of moves from the initial state to the goal state.

Algorithm

1. Represent the puzzle as a 3×3 matrix.
2. Define the goal state.
3. Locate the empty tile (0).
4. Generate all possible moves (up, down, left, right).
5. Use Breadth-First Search (BFS) to explore possible states.
6. Keep track of visited states to avoid repeated exploration.
7. If the goal state is reached, display the solution path.
8. If all possibilities are exhausted, report that no solution exists.

p5.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p5.AI (3.14.0)

File Edit Format Run Options Window Help

```
from collections import deque

def solve_puzzle(start, goal):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        state_tuple = tuple(state)
        if state_tuple in visited:
            continue

        visited.add(state_tuple)

        zero = state.index(0)
        row = zero // 3
        col = zero % 3

        moves = []

        if row > 0:
            moves.append(zero - 3)

        if row < 2:
            moves.append(zero + 3)

        if col > 0:
            moves.append(zero - 1)

        if col < 2:
            moves.append(zero + 1)

        for move in moves:
            new_state = state.copy()
            new_state[zero], new_state[move] = \
                new_state[move], new_state[zero]

            queue.append((new_state, path + [state]))

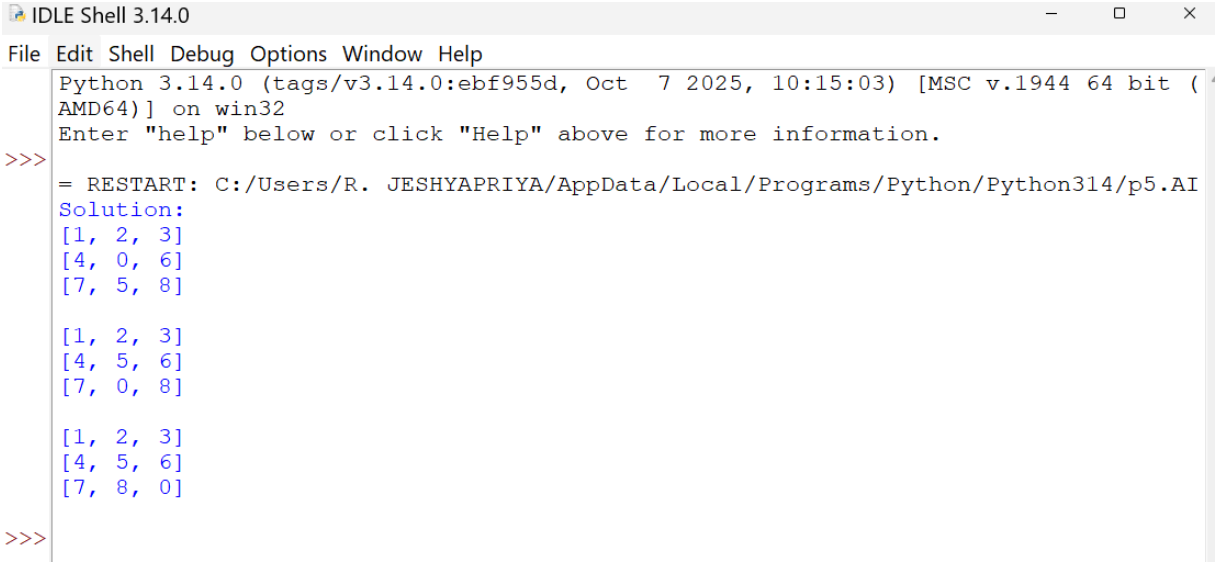
    return None


start = [1, 2, 3,
         4, 0, 6,
         7, 5, 8]

goal = [1, 2, 3,
        4, 5, 6,
        7, 8, 0]

solution = solve_puzzle(start, goal)

if solution:
    print("Solution:")
    for state in solution:
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
        print()
else:
    print("No solution exists")
```



RESULT

Thus, the 8-Puzzle Problem solved successfully using Python.

Program 2: Python Program to Solve 8-Queens Problem

Aim

To develop a Python program that solves the **8-Queens Problem** using the **Backtracking algorithm** to place 8 queens on an 8×8 chessboard such that no two queens attack each other.

Algorithm

1. Represent the chessboard as an 8×8 matrix.
2. Start placing queens row by row.
3. Check whether the current position is safe.
4. A queen must not be in the same column as another queen.
5. A queen must not be on the same diagonal as another queen.
6. If the position is safe, place the queen.
7. If no safe position is available, backtrack to the previous row.
8. Continue until all 8 queens are placed successfully.
9. Display the final chessboard solution.

p1.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p1.AI (3.14.0)

File Edit Format Run Options Window Help

```
N = 8

board = [-1] * N

def is_safe(row, col):
    for i in range(row):
        if board[i] == col or abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve(row):
    if row == N:
        return True

    for col in range(N):
        if is_safe(row, col):
            board[row] = col

            if solve(row + 1):
                return True

            board[row] = -1

    return False

if solve(0):
    print("8-Queens Solution:")

    for row in range(N):
        for col in range(N):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No solution")
```

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p1.AI
8-Queens Solution:
Q . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
>>>
```

Result

Thus, the 8-Queens Problem was solved successfully using Python and the Backtracking algorithm.

Program 3: Python Program to Solve Water Jug Problem

Aim

To develop a Python program that solves the **Water Jug Problem** using a **Breadth-First Search (BFS)** algorithm to find a sequence of operations for obtaining the required amount of water.

Algorithm

1. Represent the state using the amount of water in each jug.
2. Define the initial state as (0, 0).
3. Define the goal amount of water.
4. Generate possible operations:
    - Fill the first jug.
    - Fill the second jug.
    - Empty the first jug.
    - Empty the second jug.
    - Pour water from the first jug to the second jug.
    - Pour water from the second jug to the first jug.
5. Use **BFS** to explore all possible states.
6. Keep track of visited states to avoid repeated states.
7. If the goal state is reached, display the sequence of operations.
8. If all possibilities are exhausted, report that no solution exists.

p2.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p2.AI (3.14.0)

File Edit Format Run Options Window Help

```
def water_jug(jug1, jug2, target):
    queue = deque([((0, 0), [])])
    visited = set()

    while queue:
        (a, b), path = queue.popleft()

        if (a, b) in visited:
            continue

        visited.add((a, b))

        if a == target or b == target:
            print("Solution:")
            for step in path:
                print(step)
            print("Final State:", (a, b))
            return

        states = [
            ((jug1, b), "Fill Jug 1"),
            ((a, jug2), "Fill Jug 2"),
            ((0, b), "Empty Jug 1"),
            ((a, 0), "Empty Jug 2")
        ]

        pour = min(a, jug2 - b)
        states.append(((a - pour, b + pour), "Pour Jug 1 -> Jug 2"))

        pour = min(b, jug1 - a)
        states.append(((a + pour, b - pour), "Pour Jug 2 -> Jug 1"))

        for state, action in states:
            queue.append((state, path + [action]))

    print("No solution exists")


water_jug(4, 3, 2)
```

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p2.AI
Solution:
Fill Jug 2
Pour Jug 2 -> Jug 1
Fill Jug 2
Pour Jug 2 -> Jug 1
Final State: (4, 2)
>>>
```

Result

Thus, the Water Jug Problem was solved successfully using Python and the Breadth-First Search (BFS) algorithm.

Program 4: Python Program to Solve Crypt-Arithmetic Problem

Aim

To develop a Python program that solves a **Crypt-Arithmetic Problem** by assigning unique digits to letters such that the given arithmetic equation is satisfied.

Algorithm

1. Take the words in the crypt-arithmetic equation.
2. Identify all unique letters.
3. Assign a different digit to each letter.
4. Ensure that the first letter of each word is not zero.
5. Convert the words into numbers using the assigned digits.
6. Check whether the arithmetic equation is satisfied.
7. If the equation is satisfied, display the solution.
8. Otherwise, try another digit assignment.

p3.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p3.AI (3.14.0)

File Edit Format Run Options Window Help

```
from itertools import permutations

letters = "SENDMORY"

for p in permutations(range(10), 8):
    S, E, N, D, M, O, R, Y = p

    if S == 0 or M == 0:
        continue

    SEND = 1000*S + 100*E + 10*N + D
    MORE = 1000*M + 100*O + 10*R + E
    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

    if SEND + MORE == MONEY:
        print("Solution:")
        print("S =", S)
        print("E =", E)
        print("N =", N)
        print("D =", D)
        print("M =", M)
        print("O =", O)
        print("R =", R)
        print("Y =", Y)

        print("\nSEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
        break
```

```
>>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p3.AI
Solution:
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2

SEND = 9567
MORE = 1085
MONEY = 10652
>>>
```

Result

Thus, the Crypt-Arithmetic Problem was solved successfully using Python by assigning unique digits to each letter.

Program 5: Python Program for Missionaries and Cannibals Problem

Aim

To develop a Python program that solves the **Missionaries and Cannibals Problem** using **Breadth-First Search (BFS)** while ensuring that the missionaries are never outnumbered by cannibals.

Algorithm

1. Represent the state as (M, C, B), where M is missionaries, C is cannibals, and B represents the boat side.
2. Start with (3, 3, 0), where everyone is on the left side.
3. The boat can carry one or two people.
4. Generate all possible valid moves.
5. Check that missionaries are not outnumbered by cannibals on either side.
6. Use BFS to explore the possible states.
7. Keep track of visited states to avoid repetition.
8. Continue until the goal state (0, 0, 1) is reached.
9. Display the solution path.

p4.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p4.AI (3.14.0)

File Edit Format Run Options Window Help

```
from collections import deque

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False

    if m > 0 and m < c:
        return False

    if (3 - m) > 0 and (3 - m) < (3 - c):
        return False

    return True


def solve():
    start = (3, 3, 0)
    goal = (0, 0, 1)

    queue = deque([(start, [])])
    visited = set()

    moves = [
        (1, 0), (2, 0),
        (0, 1), (0, 2),
        (1, 1)
    ]

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue

        visited.add(state)


       if state == goal:
            print("Solution:")
            for step in path:
                print(step)
            print("Goal State:", state)
            return

       m, c, boat = state

       for dm, dc in moves:
            if boat == 0:
                new_state = (m - dm, c - dc, 1)
                action = f"{dm} Missionary, {dc} Cannibal ->"
            else:
                new_state = (m + dm, c + dc, 0)
                action = f"{dm} Missionary, {dc} Cannibal <-"

            if is_valid(new_state[0], new_state[1]):
                queue.append((new_state, path + [new_state]))

   print("No solution exists")


olve()
```

```
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p4.AI
Solution:
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
Goal State: (0, 0, 1)
```

Result

Thus, the Missionaries and Cannibals Problem was solved successfully using Python and the Breadth-First Search (BFS) algorithm.

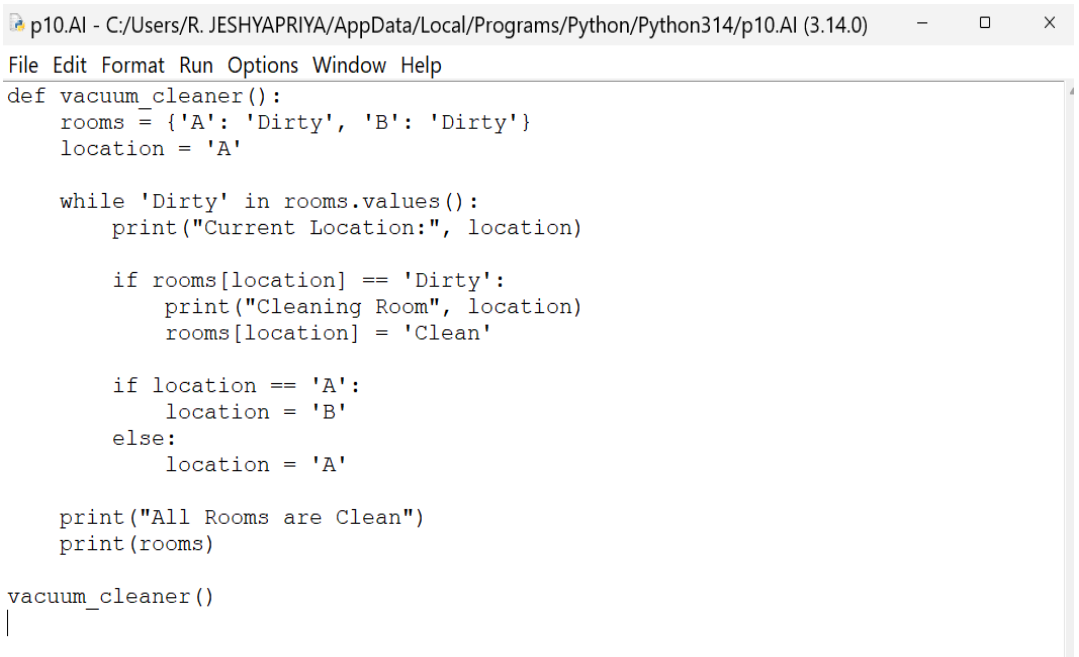
Program 6: Python Program to Solve Vacuum Cleaner Problem

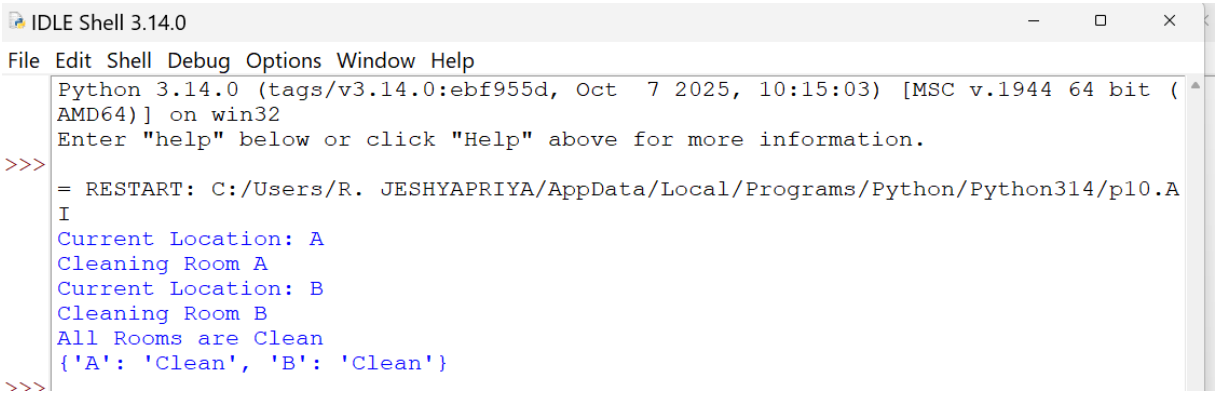
Aim

To write a Python program to solve the **Vacuum Cleaner Problem** by checking the condition of rooms and cleaning the dirty rooms.

Algorithm

1. Represent two rooms as A and B.
2. Check the condition of each room.
3. If the room is dirty, clean it.
4. If the room is clean, move to the next room.
5. Repeat until all rooms are clean.
6. Display the final status of the rooms.





Result

Thus, the **Vacuum Cleaner Problem was solved successfully using Python** by cleaning all the dirty rooms.

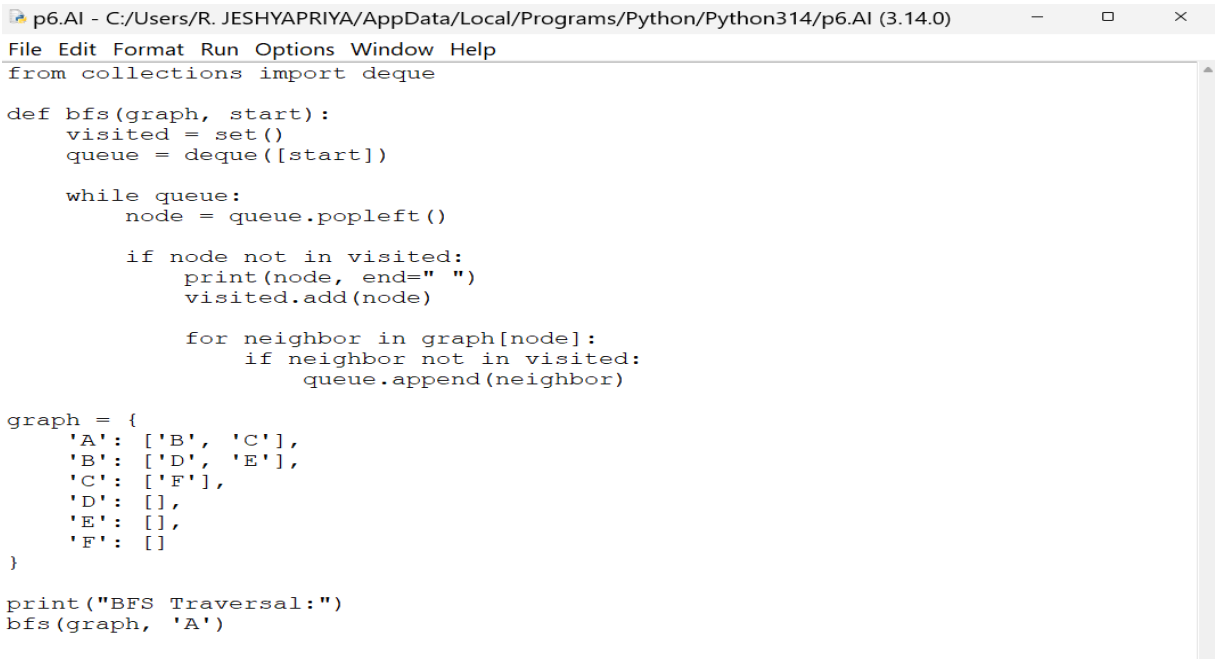
Program 7: Python Program to Implement BFS (Breadth-First Search)

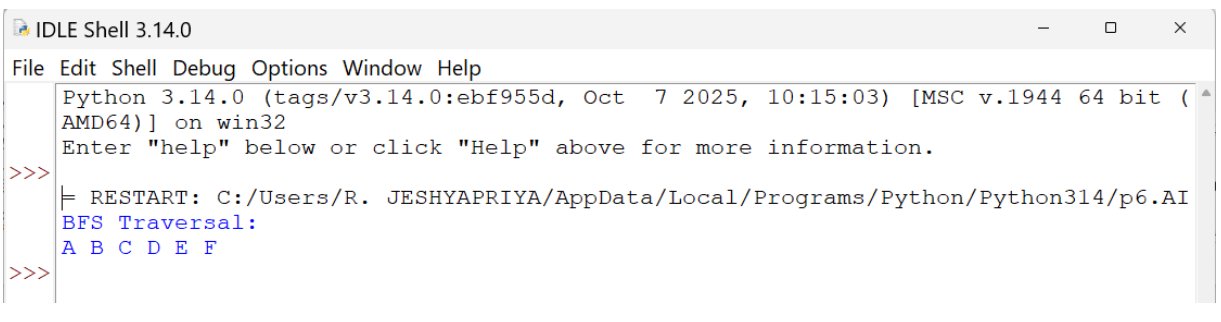
Aim

To write a Python program to implement **Breadth-First Search (BFS)** to traverse a graph.

Algorithm

1. Create a graph using an adjacency list.
2. Initialize a queue with the starting node.
3. Mark the starting node as visited.
4. Remove a node from the front of the queue and print it.
5. Add all unvisited neighboring nodes to the queue.
6. Repeat until the queue becomes empty.





Result

Thus, the **Breadth-First Search (BFS)** algorithm was implemented successfully using Python.

Program 8: Python Program to Implement DFS (Depth-First Search)

Aim

To write a Python program to implement **Depth-First Search (DFS)** to traverse a graph.

Algorithm

1. Create a graph using an adjacency list.
2. Start from the initial node.
3. Mark the node as visited and print it.
4. Visit an unvisited neighboring node.
5. Continue until no unvisited neighbors remain.
6. Backtrack and visit other unvisited nodes.
7. Repeat until all reachable nodes are visited.

p7.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p7.AI (3.14.0)

File Edit Format Run Options Window Help

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

print("DFS Traversal:")
dfs(graph, 'A', set())
```

```
>>>
    = RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p7.AI
    DFS Traversal:
    A B D E C F
>>>
```

Result

Thus, the **Depth-First Search (DFS)** algorithm was implemented successfully using Python.

Program 9: Python Program to Solve Travelling Salesman Problem (TSP)

Aim

To write a Python program to solve the **Travelling Salesman Problem (TSP)** by finding the shortest possible route that visits every city exactly once and returns to the starting city.

Algorithm

1. Represent the cities and distances using a cost matrix.
2. Select the starting city.
3. Generate all possible routes to visit the remaining cities.
4. Calculate the total distance for each route.
5. Select the route with the minimum distance.
6. Display the shortest route and its total distance.

```
p8.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p8.AI (3.14.0)
File  Edit  Format  Run  Options  Window  Help
from itertools import permutations

def tsp(graph, start):
    cities = list(range(len(graph)))
    cities.remove(start)

    min_cost = float('inf')
    best_path = []

    for path in permutations(cities):
        route = [start] + list(path) + [start]
        cost = 0

        for i in range(len(route) - 1):
            cost += graph[route[i]][route[i + 1]]

        if cost < min_cost:
            min_cost = cost
            best_path = route

    print("Shortest Path:", best_path)
    print("Minimum Cost:", min_cost)


graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

tsp(graph, 0)
```

```
>>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p8.AI
Shortest Path: [0, 1, 3, 2, 0]
Minimum Cost: 80
>>>
```

Result

Thus, the **Travelling Salesman Problem was solved successfully using Python**, finding the shortest route that visits every city exactly once and returns to the starting city.

Program 10: Python Program to Implement A* Algorithm

Aim

To write a Python program to implement the *A (A-Star) Search Algorithm** to find the shortest path from the start node to the goal node using path cost and heuristic cost.

Algorithm

1. Define the graph with nodes and edge costs.
2. Assign a heuristic value to each node.
3. Initialize the start node.
4. Calculate the total cost using f(n) = g(n) + h(n).
5. Select the node with the lowest total cost.
6. Explore its neighboring nodes and update their costs.
7. Repeat until the goal node is reached.
8. Display the shortest path and its total cost.

p9.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p9.AI (3.14.0)

File Edit Format Run Options Window Help

```
def astar(graph, heuristic, start, goal):
    open_list = [(start, 0)]
    cost = {start: 0}
    parent = {}

    while open_list:
        open_list.sort(key=lambda x: x[1])
        node, _ = open_list.pop(0)

        if node == goal:
            path = []
            while node in parent:
                path.append(node)
                node = parent[node]
            path.append(start)
            path.reverse()

            print("Shortest Path:", path)
            print("Total Cost:", cost[goal])
            return

        for neighbor, distance in graph[node]:
            new_cost = cost[node] + distance

            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                priority = new_cost + heuristic[neighbor]
                parent[neighbor] = node
                open_list.append((neighbor, priority))

    print("No path found")
```

```
graph = {
     'A': [('B', 1), ('C', 4)],
     'B': [('D', 2), ('E', 5)],
     'C': [('F', 3)],
     'D': [('G', 5)],
     'E': [('G', 2)],
     'F': [('G', 2)],
     'G': []
}

heuristic = {
     'A': 7, 'B': 6, 'C': 4,
     'D': 4, 'E': 2, 'F': 1, 'G': 0
}

astar(graph, heuristic, 'A', 'G')
```

```
>>>
    = RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p9.AI
    Shortest Path: ['A', 'B', 'D', 'G']
    Total Cost: 8
>>>
```

Result

Thus, the *A Search Algorithm was implemented successfully using Python** to find the shortest path from the start node to the goal node.

Program 11: Python Program for Map Coloring Using CSP

Aim

To write a Python program to solve the **Map Coloring Problem using Constraint Satisfaction Problem (CSP)** by assigning colors to regions so that no two adjacent regions have the same color.

Algorithm

1. Define the regions of the map.
2. Define the colors available.
3. Specify the neighboring regions.
4. Assign a color to each region.
5. Check whether adjacent regions have different colors.
6. If a conflict occurs, try another color using backtracking.
7. Continue until all regions are colored successfully.
8. Display the final color assignment.

p11.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p11.AI (3.14.0)

File Edit Format Run Options Window Help

```
def is_safe(region, color, colors, neighbors):
    for neighbor in neighbors[region]:
        if colors.get(neighbor) == color:
            return False
    return True


def solve(regions, color_list, neighbors, colors):
    if len(colors) == len(regions):
        return True

    region = next(r for r in regions if r not in colors)

    for color in color_list:
        if is_safe(region, color, colors, neighbors):
            colors[region] = color

            if solve(regions, color_list, neighbors, colors):
                return True

            del colors[region]

    return False
```

```
regions = ['A', 'B', 'C', 'D']

color_list = ['Red', 'Green', 'Blue']

neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colors = {}

if solve(regions, color_list, neighbors, colors):
    print("Map Coloring Solution:")
    for region in regions:
        print(region, ":", colors[region])
else:
    print("No solution exists")
```

```
>>>
    = RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Py
    thon314/p11.AI
    Map Coloring Solution:
    A : Red
    B : Green
    C : Blue
    D : Red
>>>
```

Result

Thus, the Map Coloring Problem was solved successfully using a Constraint Satisfaction Problem (CSP) and the Backtracking algorithm.

Program 12: Python Program to Implement Tic-Tac-Toe Game

Aim

To write a Python program to implement the **Tic-Tac-Toe game** between two players using a 3×3 board.

Algorithm

1. Create a 3×3 game board.
2. Assign X and O to two players.
3. Display the game board.
4. Ask the current player to enter a position.
5. Check whether the position is empty.
6. Check whether a player has won by completing a row, column, or diagonal.
7. If all positions are filled without a winner, declare a draw.
8. Repeat until the game ends.

p12.py - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p12.py (3.14.0)

File Edit Format Run Options Window Help

```
def display(board):
    for row in board:
        print(" | ".join(row))
        print("-" * 9)


def check_winner(board, player):
    for i in range(3):
        if all(board[i][j] == player for j in range(3)):
            return True

        if all(board[j][i] == player for j in range(3)):
            return True

    if all(board[i][i] == player for i in range(3)):
        return True

    if all(board[i][2-i] == player for i in range(3)):
        return True

    return False


board = [[" " for _ in range(3)] for _ in range(3)]
player = "X"
```

```python
while True:
    display(board)

    row = int(input("Enter row (0-2): "))
    col = int(input("Enter column (0-2): "))

    if row not in range(3) or col not in range(3):
        print("Invalid position! Try again.")
        continue

    if board[row][col] != " ":
        print("Position already occupied! Try again.")
        continue

    board[row][col] = player

    if check_winner(board, player):
        display(board)
        print("Player", player, "wins!")
        break

    if all(board[i][j] != " " for i in range(3) for j in range(3)):
        display(board)
        print("Match Draw!")
        break

    player = "O" if player == "X" else "X"
```

```
>>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p12.py
   |   |
---------
   |   |
---------
   |   |
---------
Enter row (0-2): 0
Enter column (0-2): 0
X |   |
---------
   |   |
---------
   |   |
---------
Enter row (0-2): 0
Enter column (0-2): 1
X | O |
---------
   |   |
---------
   |   |
---------
Enter row (0-2): 1
Enter column (0-2): 1
X | O |
---------
   | X |
---------
   |   |
---------
```

```
Enter row (0-2): 0
Enter column (0-2): 2
X | O | O
---------
  | X |
---------
  |   |
---------
Enter row (0-2): 2
Enter column (0-2): 2
X | O | O
---------
  | X |
---------
  |   | X
---------
Player X wins!
```

Result

Thus, the **Tic-Tac-Toe game was implemented successfully using Python**, allowing two players to play alternately and identifying the winner or draw.

Program 13: Python Program to Implement Minimax Algorithm for Gaming

Aim

To write a Python program to implement the **Minimax Algorithm** to find the optimal move in a two-player game.

Algorithm

1. Define the game tree with possible moves.
2. Assign scores to the terminal nodes.
3. MAX player selects the maximum score.
4. MIN player selects the minimum score.
5. Apply the Minimax algorithm recursively.
6. Continue until the root node is evaluated.
7. Display the best possible score.

p13.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p13.AI (3.14.0)

File Edit Format Run Options Window Help

```
def minimax(depth, node, is_max, scores, height):
    if depth == height:
        return scores[node]

    if is_max:
        return max(
            minimax(depth + 1, node * 2, False, scores, height),
            minimax(depth + 1, node * 2 + 1, False, scores, height)
        )
    else:
        return min(
            minimax(depth + 1, node * 2, True, scores, height),
            minimax(depth + 1, node * 2 + 1, True, scores, height)
        )


scores = [3, 5, 2, 9, 12, 5, 23, 23]
height = 3

result = minimax(0, 0, True, scores, height)

print("Best Score:", result)
```

```
>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Py
thon314/p13.AI
Best Score: 12
>>
```

Result

Thus, the **Minimax Algorithm was implemented successfully using Python** to determine the optimal score in a two-player game.

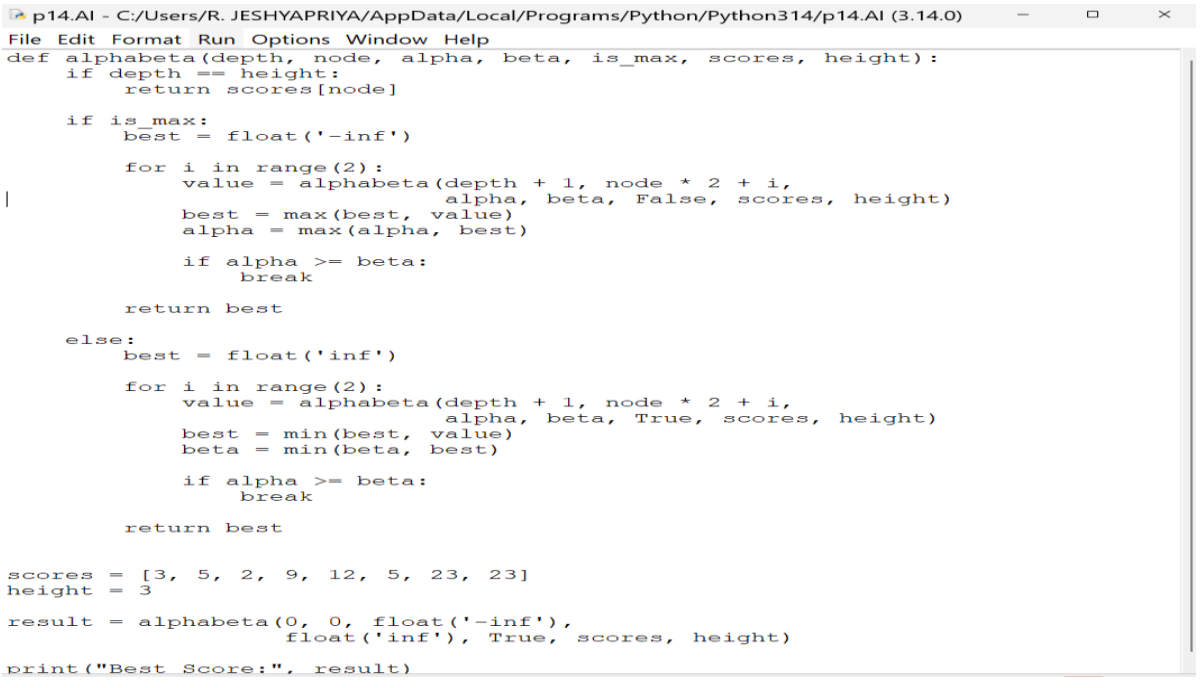
Program 14: Python Program to Implement Alpha-Beta Pruning Algorithm for Gaming

Aim

To write a Python program to implement the **Alpha-Beta Pruning Algorithm** to find the optimal move in a two-player game by reducing the number of nodes evaluated in the Minimax algorithm.

Algorithm

1. Define the game tree with possible scores.
2. Initialize alpha as negative infinity and beta as positive infinity.
3. MAX player selects the maximum score.
4. MIN player selects the minimum score.
5. Update alpha and beta values at each node.
6. If alpha is greater than or equal to beta, prune the remaining branches.
7. Display the best possible score.



```
>>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Py
thon314/p14.AI
Best Score: 12
>>>
```

Result

Thus, the Alpha-Beta Pruning algorithm was implemented successfully using Python to find the optimal score while pruning unnecessary branches.

Program 15: Python Program to Implement Decision Tree

Aim

To write a Python program to implement a **Decision Tree Classification Algorithm** to classify data based on input features.

Algorithm

1. Import the required libraries.
2. Create a dataset with input features and output labels.
3. Separate the features and target values.
4. Create a Decision Tree Classifier.
5. Train the model using the dataset.
6. Predict the output for new input data.
7. Display the predicted result.

p15.AI - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p15.AI (3.14.0)

File Edit Format Run Options Window Help

```
from sklearn.tree import DecisionTreeClassifier


X = [[1, 1], [1, 0], [0, 1], [0, 0]]
y = [1, 1, 0, 0]


model = DecisionTreeClassifier()


model.fit(X, y)


result = model.predict([[1, 1]])

print("Predicted Output:", result[0])
```

```
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p15.A
I
Predicted Output: 1
>>>
```

Result

Thus, the **Decision Tree Classification Algorithm was implemented successfully using Python** to predict the output based on input features.

16.Program: Python Program to Implement Feed Forward Neural Network

Aim

To write a Python program to implement a **Feed Forward Neural Network (FFNN)** to predict the output from given input data.

Algorithm

1. Import the required libraries.
2. Create the input dataset and target output.
3. Initialize the neural network model.
4. Add input, hidden, and output layers.
5. Train the model using the dataset.
6. Predict the output for new input data.
7. Display the predicted output.

p16.py - C:/Users/R. JESHYAPRIYA/AppData/Local/Programs/Python/Python314/p16.py (3.14.0)

File Edit Format Run Options Window Help

```
from sklearn.neural_network import MLPClassifier

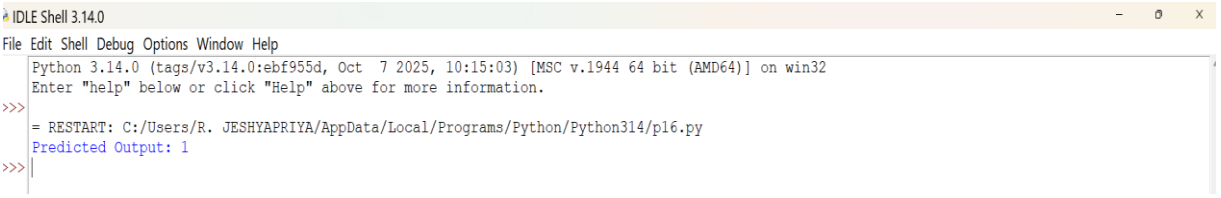

X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 1]


model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='lbfgs',
    max_iter=1000,
    random_state=1
)


model.fit(X, y)


result = model.predict([[1, 1]])

print("Predicted Output:", result[0])
```



Result

Thus, the **Feed Forward Neural Network was implemented successfully using Python** to predict the output from the given input data.

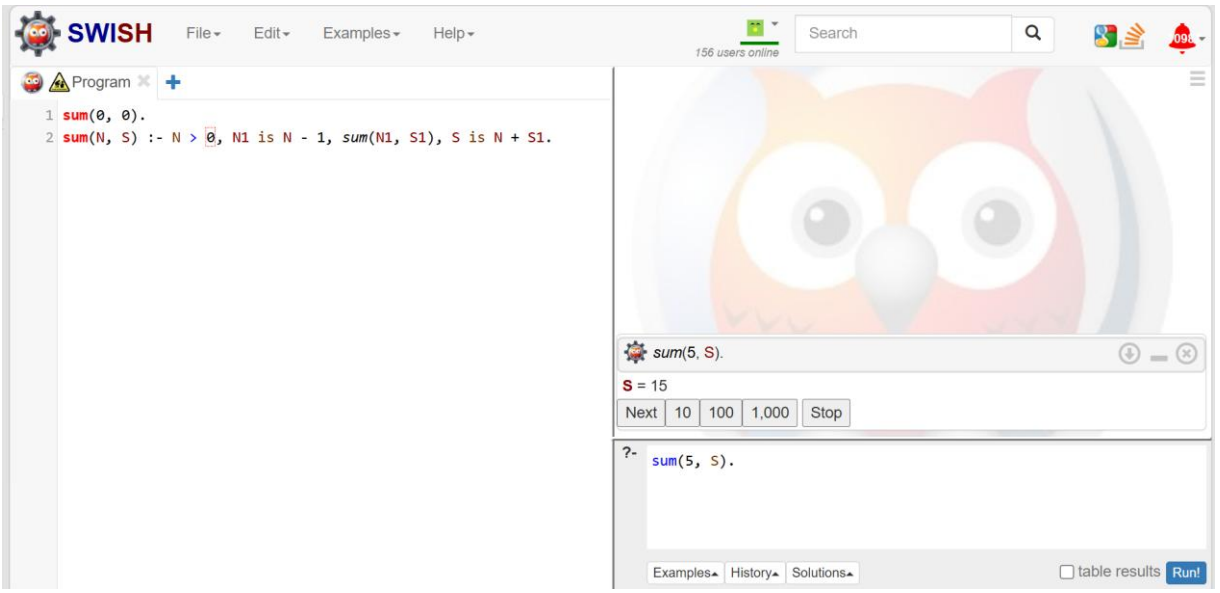
17.Program: Prolog Program to Sum Integers from 1 to N

Aim

To write a Prolog program to calculate the sum of integers from 1 to N using recursion.

Algorithm

1. Read the value of N.
2. If N is 0, return 0.
3. Otherwise, calculate the sum of N and the sum of integers from 1 to N−1.
4. Display the result.



Result

Thus, the Prolog program to calculate the sum of integers from 1 to N was implemented successfully using recursion.

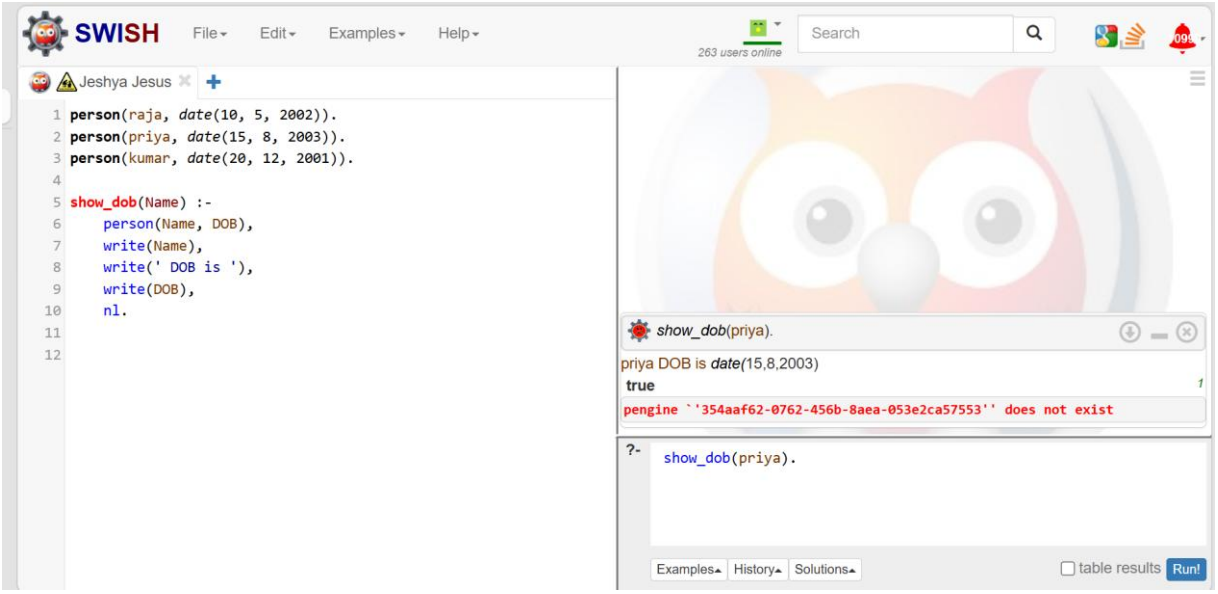
Program 18: Prolog Program for a Database with Name and DOB

Aim

To write a Prolog program to create a database containing names and dates of birth (DOB) and retrieve the details.

Algorithm

1. Define facts containing names and dates of birth.
2. Create a rule to retrieve a person's DOB.
3. Query the database using a person's name.
4. Display the name and DOB.



Result

Thus, the Prolog program to create a database containing names and DOB was implemented successfully.

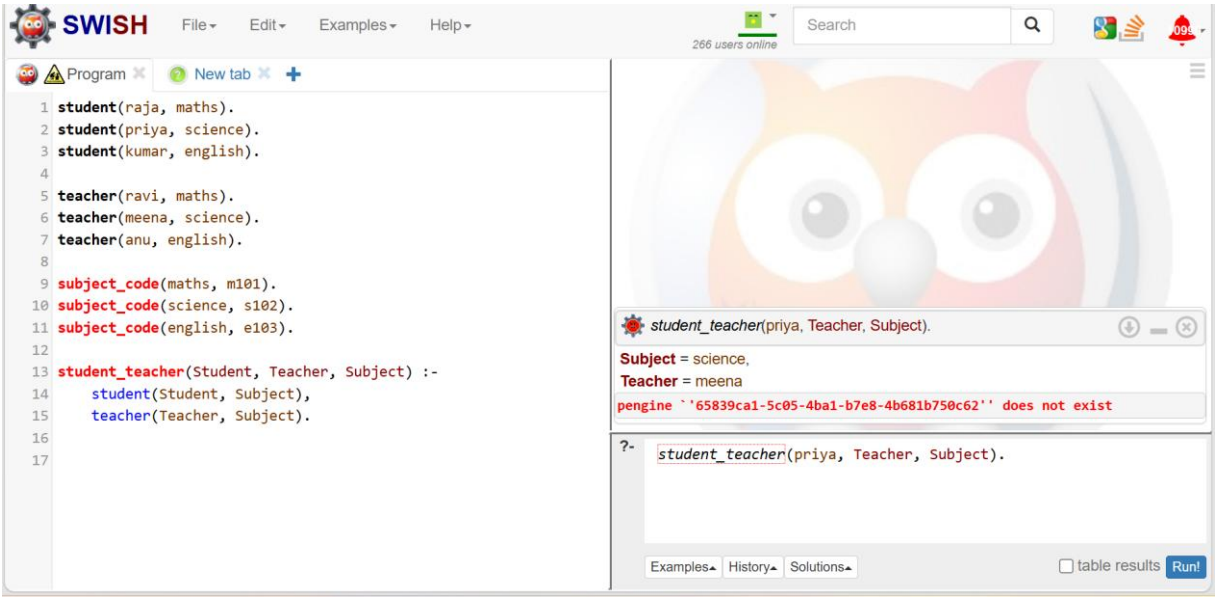
Program 19: Prolog Program for Student-Teacher-Subject Code

Aim

To write a Prolog program to create a database containing student, teacher, and subject-code details.

Algorithm

1. Define facts for students and their subjects.
2. Define facts for teachers and their subjects.
3. Define subject codes.
4. Create a rule to identify the teacher and subject for a student.
5. Query the database and display the result.



Result

Thus, the Prolog program to retrieve student, teacher, and subject details was implemented successfully.

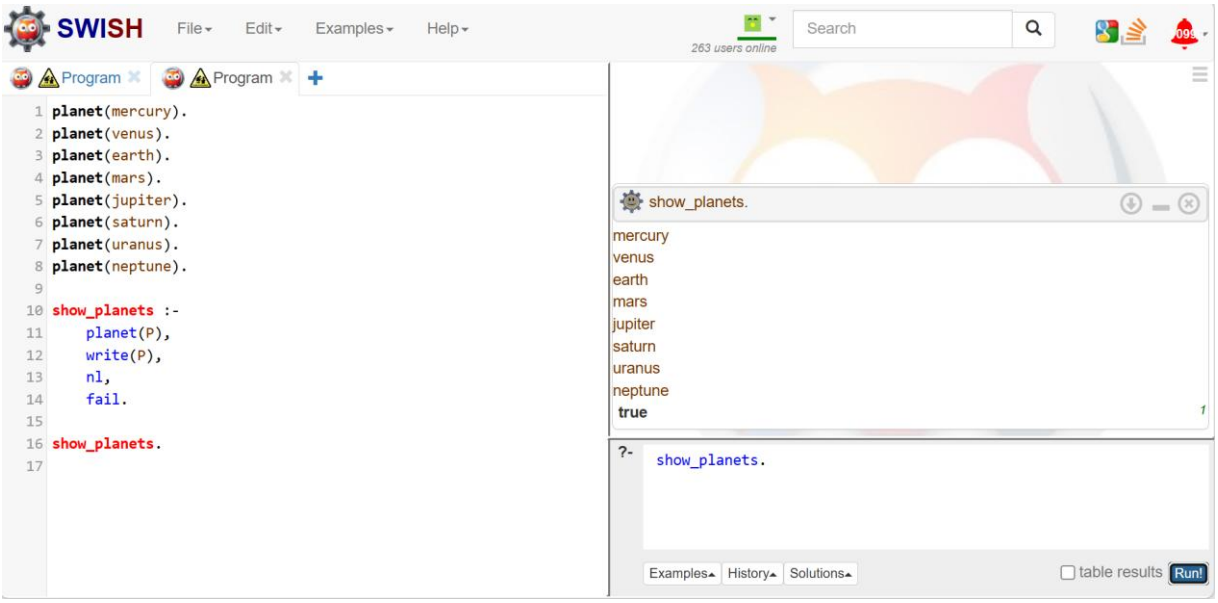
Program 20: Prolog Program for Planets Database

Aim

To write a Prolog program to create a database containing the names of the planets and display them.

Algorithm

1. Define facts for the eight planets.
2. Create a rule to retrieve the planet names.
3. Display each planet on a separate line.
4. Stop after displaying all planets.



Result

Thus, the Prolog program to create and display the planets database was implemented successfully.

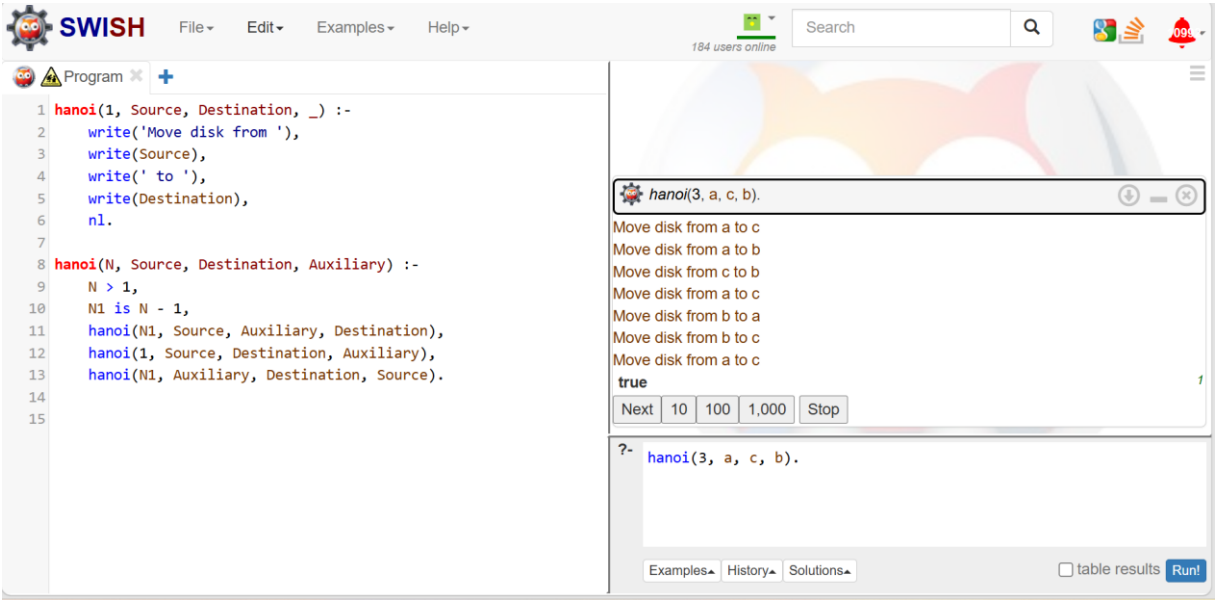
Program 21: Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to solve the **Towers of Hanoi problem** using recursion.

Algorithm

1. Start with N disks on the source rod.
2. If N = 1, move the disk from the source rod to the destination rod.
3. Move N−1 disks from the source rod to the auxiliary rod.
4. Move the largest disk from the source rod to the destination rod.
5. Move N−1 disks from the auxiliary rod to the destination rod.
6. Display all the moves.



Result

Thus, the **Towers of Hanoi problem was implemented successfully using Prolog and recursion**.

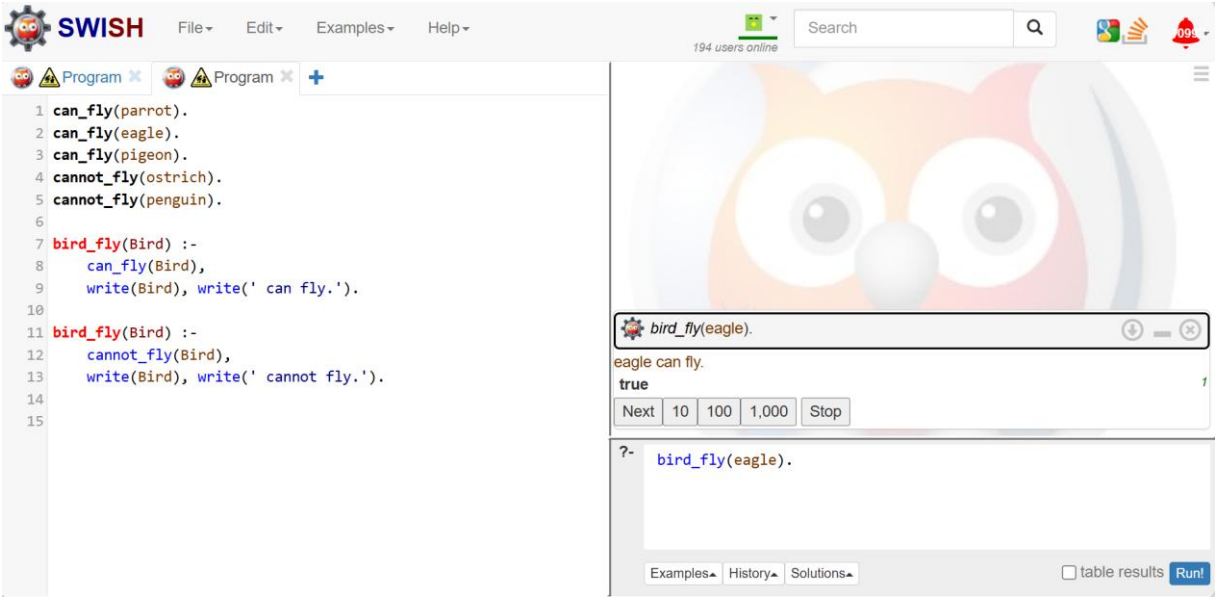
Program 22: Prolog Program to Check Whether a Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a particular bird can fly using facts and rules.

Algorithm

1. Define facts about birds that can fly.
2. Define facts about birds that cannot fly.
3. Create a rule to check whether a bird can fly.
4. Query the program with a bird's name.
5. Display the result.



Result

Thus, the Prolog program to check whether a bird can fly was implemented successfully.

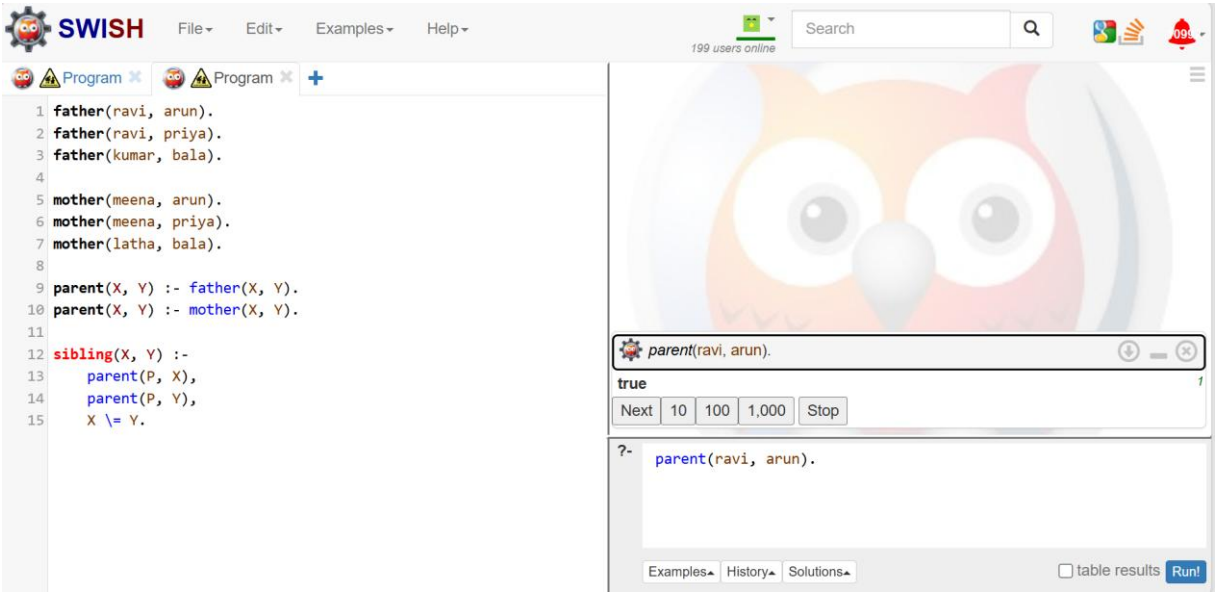
Program 23: Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree and identify parent, father, mother, and sibling relationships.

Algorithm

1. Define facts for father and mother relationships.
2. Create a rule to identify parents.
3. Create a rule to identify siblings.
4. Query the database for family relationships.
5. Display the result.



Result

Thus, the family tree was implemented successfully using Prolog facts and rules.

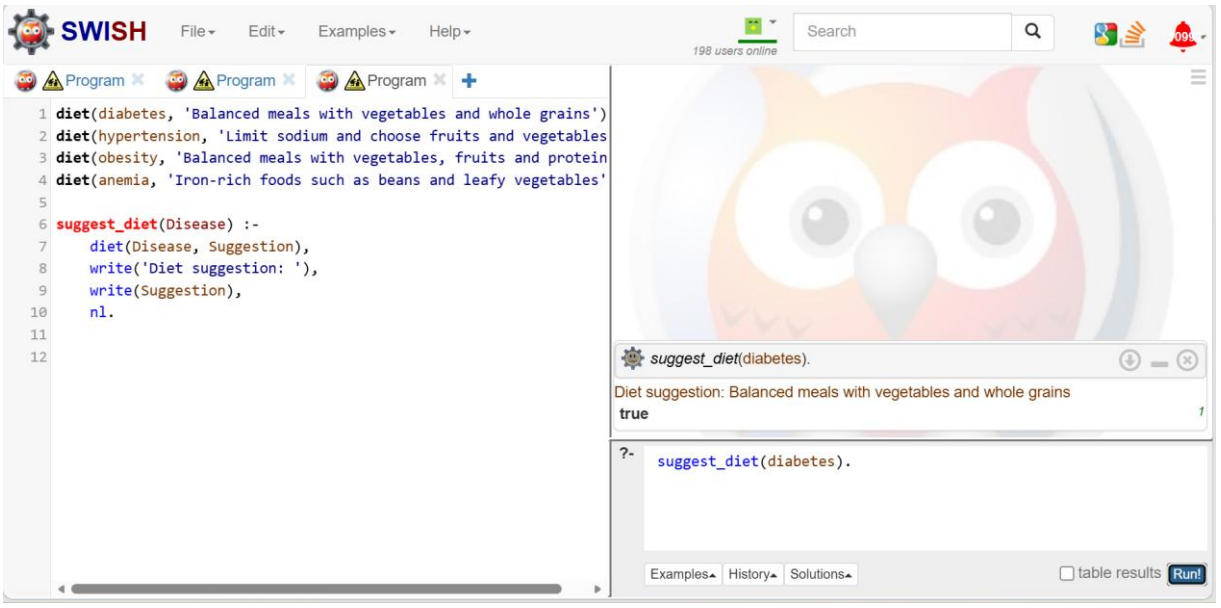
Program 24: Prolog Program to Suggest a Dieting System Based on Disease

Aim

To write a Prolog program to suggest general dietary options based on a given health condition.

Algorithm

1. Define dietary suggestions for different health conditions.
2. Create a rule to retrieve the suggestions.
3. Query the program with a condition.
4. Display the suggested dietary options.



Result

Thus, the Prolog program to provide general dietary suggestions based on a health condition was implemented successfully. These are general examples, not personalized medical advice.

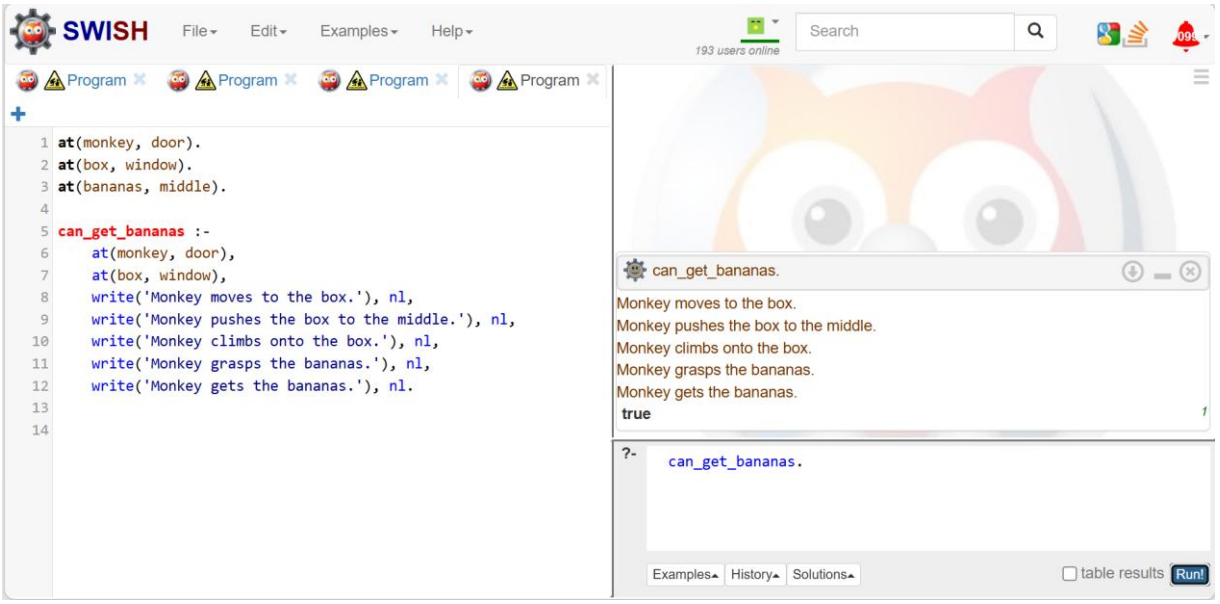
Program 25: Prolog Program to Implement Monkey Banana Problem

Aim

To write a Prolog program to solve the **Monkey Banana Problem** by determining the actions required for a monkey to obtain bananas.

Algorithm

1. Define the initial position of the monkey.
2. Define the position of the box and bananas.
3. Represent the actions: move, push the box, climb the box, and grasp bananas.
4. Define the conditions required to obtain the bananas.
5. Query the program to determine whether the monkey can get the bananas.
6. Display the solution.



Result

Thus, the Monkey Banana Problem was demonstrated successfully using a Prolog program.

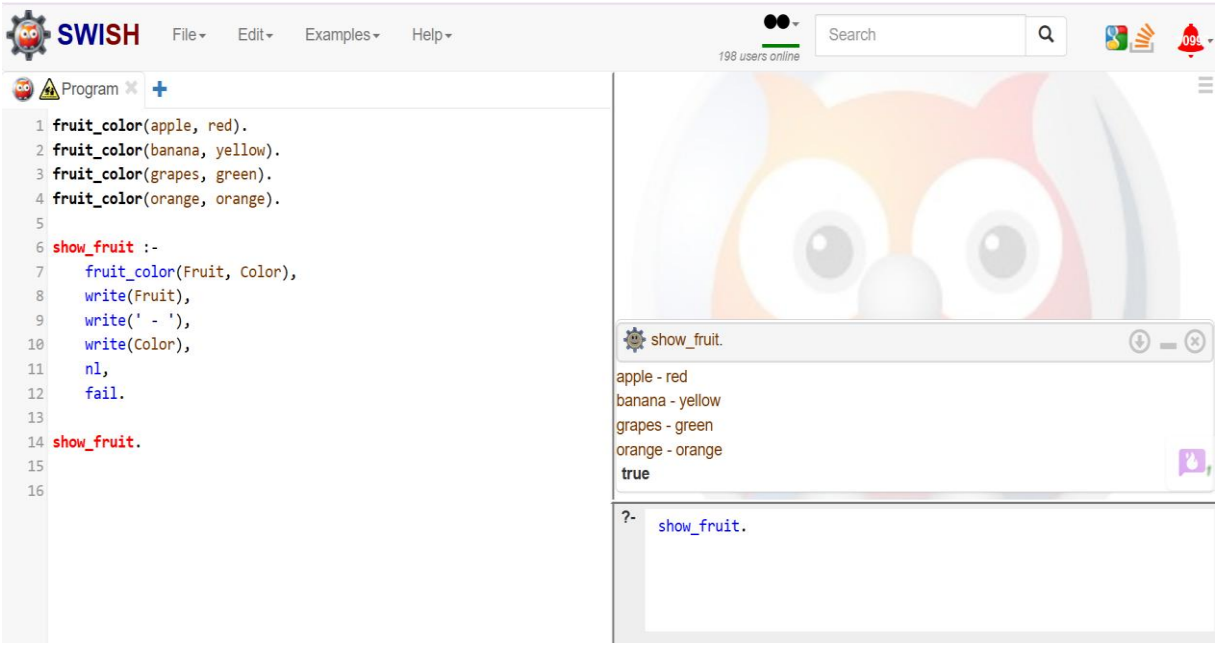
Program 26: Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write a Prolog program to identify fruits and their colors using backtracking.

Algorithm

1. Define facts for fruits and their colors.
2. Create a rule to display the fruit and its color.
3. Query the database.
4. Use backtracking to find other matching fruits.
5. Display the results.



Result

Thus, the fruit and color database was implemented successfully using Prolog backtracking.

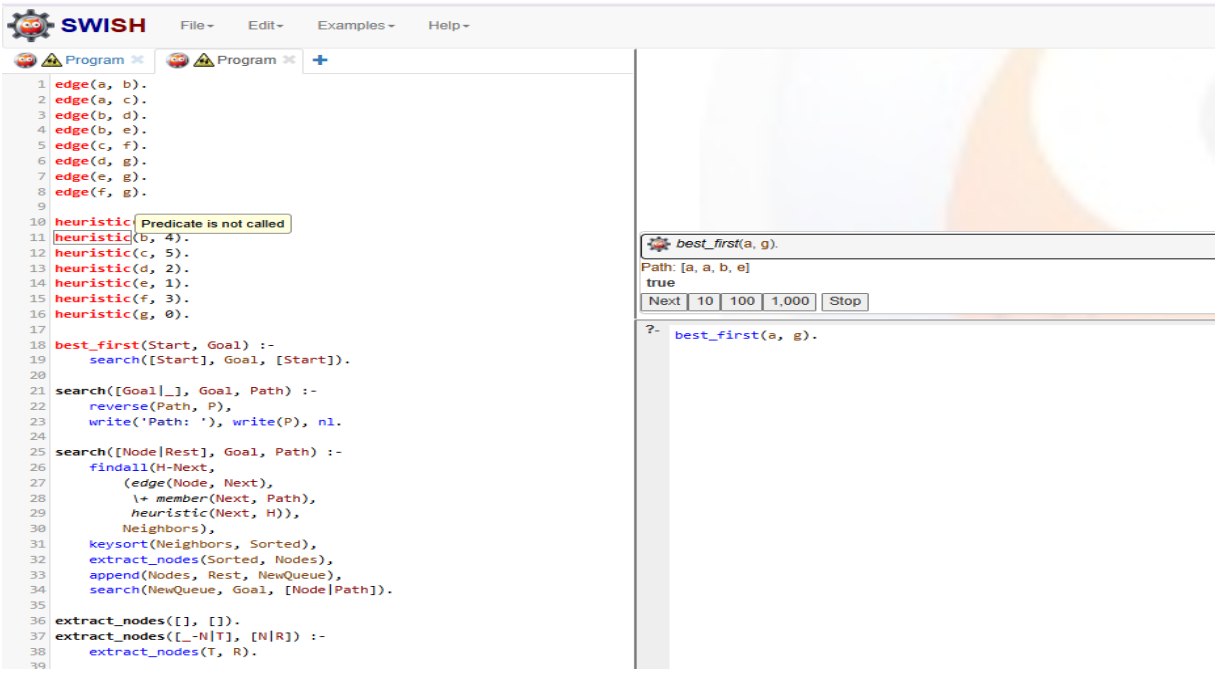
Program 27: Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm to find a path from the start node to the goal node.

Algorithm

1. Define the graph and heuristic values.
2. Initialize the starting node.
3. Select the node with the lowest heuristic value.
4. Expand the selected node and add its unvisited neighbors.
5. Repeat until the goal node is reached.
6. Display the path.



Result

Thus, the Best First Search algorithm was implemented using Prolog.

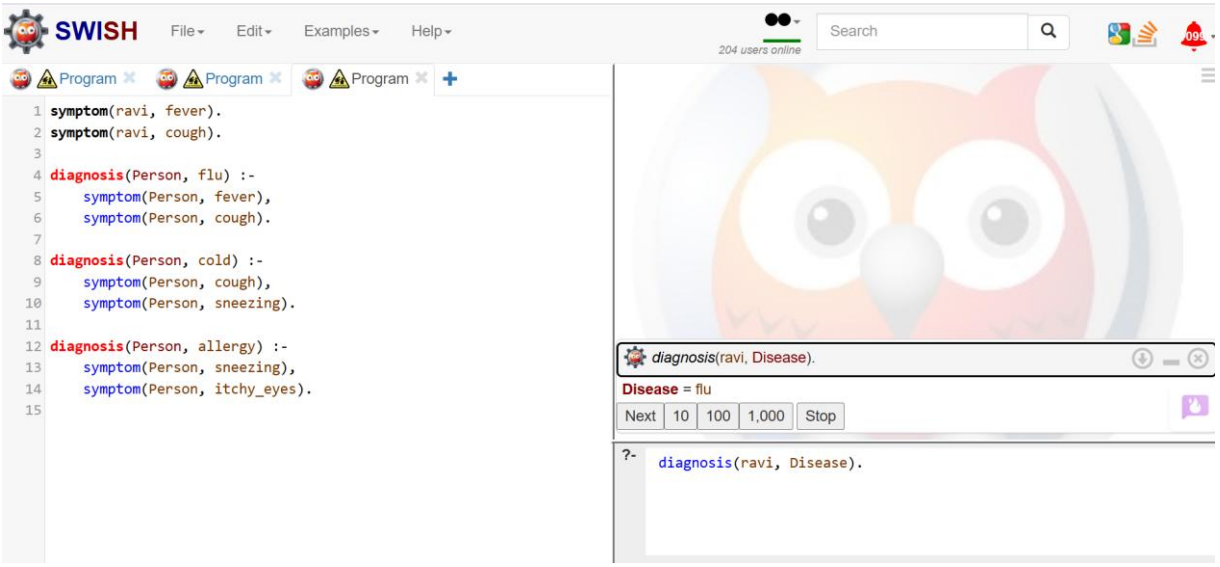
Program 28: Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to identify possible common illnesses based on symptoms.

Algorithm

1. Define symptoms associated with common illnesses.
2. Create rules for possible diagnoses.
3. Enter the symptoms.
4. Check which diagnosis matches the symptoms.
5. Display the possible diagnosis.



Result

Thus, the Prolog medical diagnosis example was implemented successfully. These rules are illustrative only and cannot provide a reliable medical diagnosis.

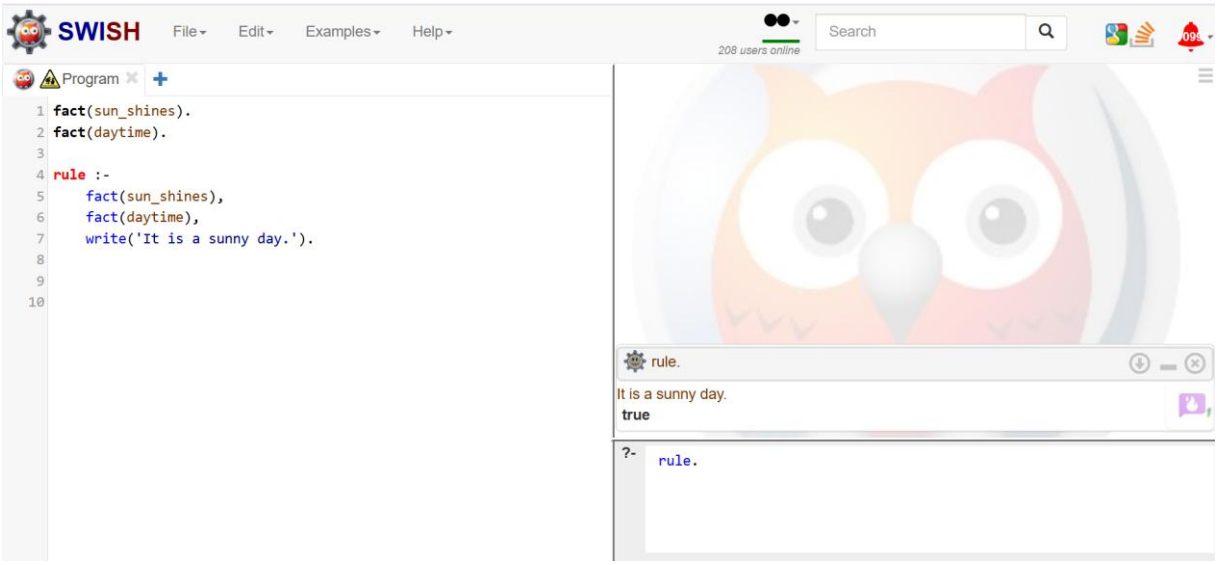
Program 29: Prolog Program for Forward Chaining

Aim

To write a Prolog program to demonstrate forward chaining by deriving new facts from existing facts and rules.

Algorithm

1. Define the initial facts.
2. Define rules based on the facts.
3. Check whether the conditions of a rule are satisfied.
4. Derive new facts when the conditions are true.
5. Display the derived result.



Result

Thus, a simple rule-based reasoning example was implemented using Prolog.

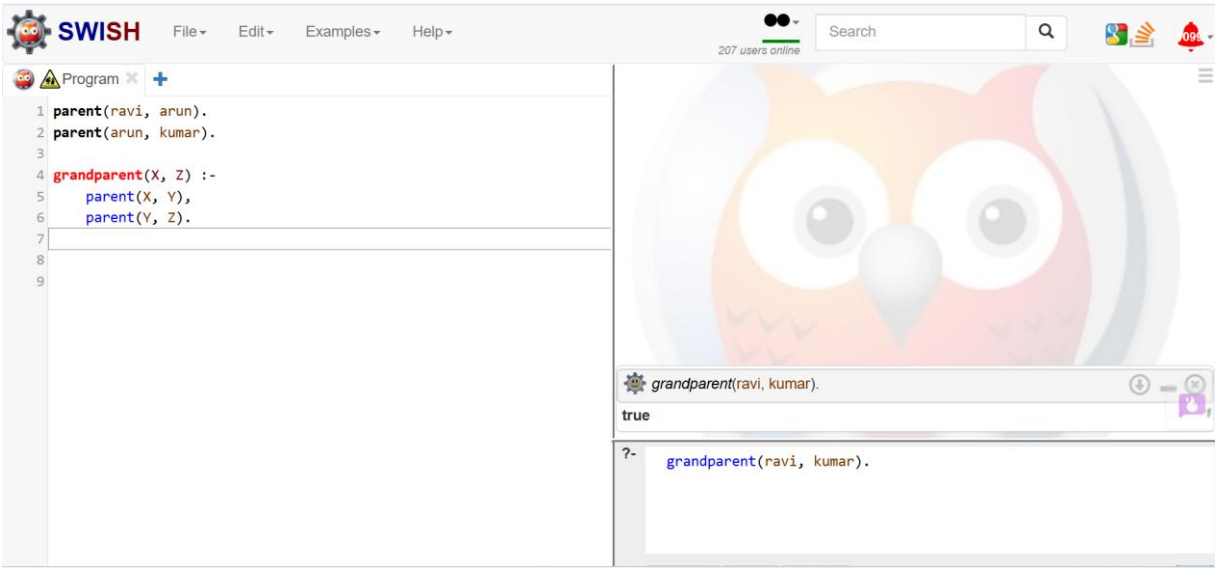
Program 30: Prolog Program for Backward Chaining

Aim

To write a Prolog program to demonstrate backward chaining by checking whether a goal can be proved using facts and rules.

Algorithm

1. Define the facts.
2. Define rules for the required goal.
3. Query the goal.
4. Prolog checks whether the facts satisfy the rule conditions.
5. Display whether the goal succeeds.



Result

Thus, backward chaining was demonstrated successfully using Prolog facts and rules.

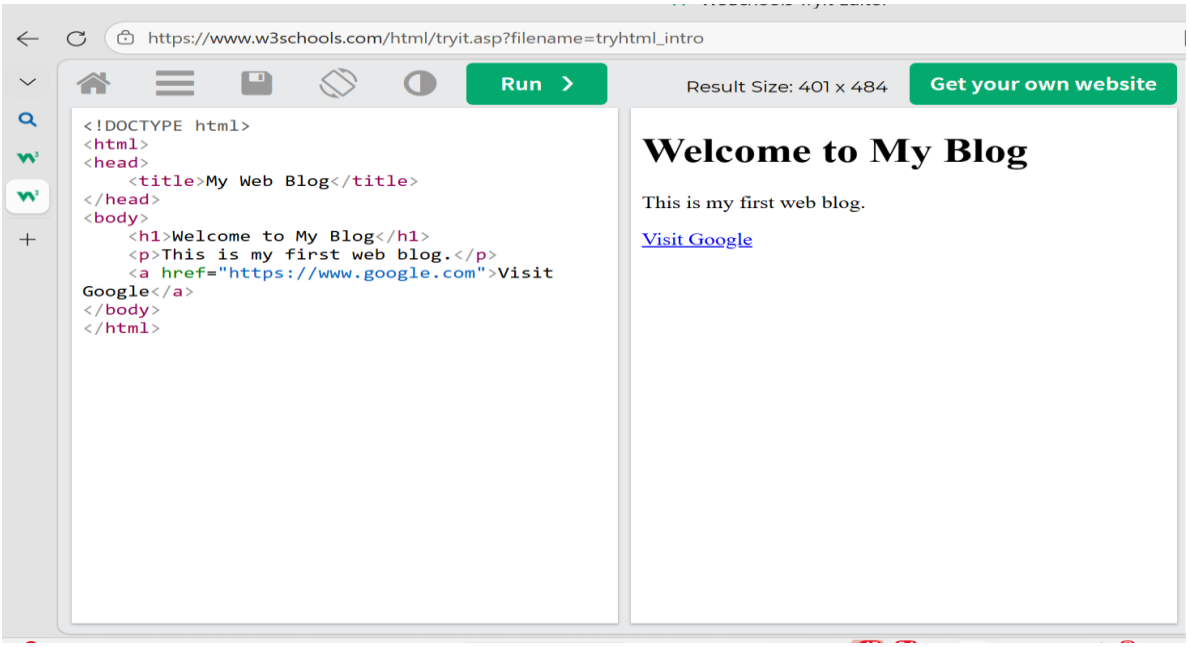
Program 31: Create a Web Blog Using WordPress to Demonstrate Anchor Tag, Title Tag, etc.

Aim

To create a web blog using WordPress and demonstrate HTML elements such as the anchor tag, title tag, and other basic HTML tags.

Algorithm

1. Open WordPress and create a new blog.
2. Enter the blog title and content.
3. Use the title tag to specify the webpage title.
4. Use the anchor tag (<a>) to create hyperlinks.
5. Add headings, paragraphs, and other HTML elements.
6. Preview and publish the blog.



Result

Thus, a web blog was created successfully, demonstrating the title tag, anchor tag, and other basic HTML elements.

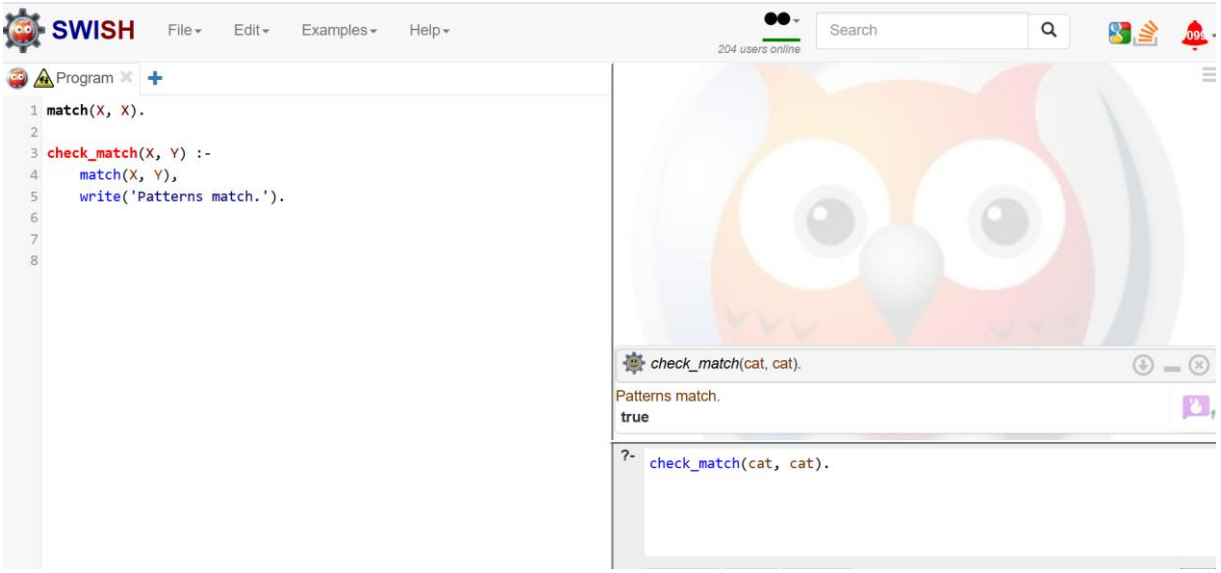
Program 32: Prolog Program to Implement Pattern Matching

Aim

To write a Prolog program to implement pattern matching by comparing two terms.

Algorithm

1. Define two terms.
2. Compare the terms using unification.
3. If they match, display a success message.
4. Otherwise, report that they do not match.



Result

Thus, pattern matching was implemented successfully using Prolog unification.

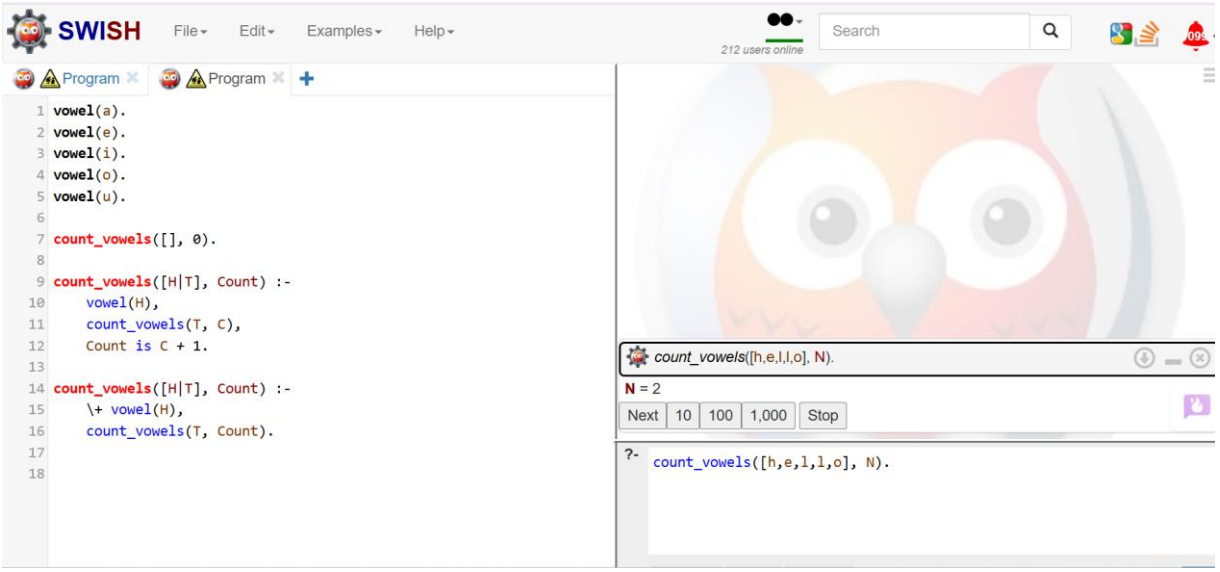
Program 33: Prolog Program to Find the Number of Vowels

Aim

To write a Prolog program to count the number of vowels in a given list of characters.

Algorithm

1. Define the vowels.
2. Read a list of characters.
3. Check each character to determine whether it is a vowel.
4. Count the vowels recursively.
5. Display the total number of vowels.



Result

Thus, the Prolog program to count vowels in a list of characters was implemented successfully.